Functions

Parameters, return values, and dynamic scoping

Gregory S. DeLozier, PhD

Chapter Route

1. Writing a function, return, and null
2. Local variables and call frames
3. Dynamic scoping, and its real limitation
4. Recursion and calls as expressions
5. Parsing, evaluation order, and the new boundary error

Functions in Vertex

- A function value is written with `function`, a parameter list, and a body.
- Functions are first-class values: they can be stored, passed, and returned like any other value.
- Calling with the wrong number of arguments is a runtime error.
- A function that falls off the end, or returns with no expression, produces `null`.

Writing a Function

```
double = function(x) { return x + x };
print(double(21));
```

Prints 42.

The Named Form Is Sugar

```
function triple(x) { return x + x + x };
print(triple(7));
```

Prints 21. Exactly `triple = function(x) { return x + x + x };`

Arity Is Checked at the Call

```
function add3(a, b, c) { return a + b + c };
add3(1, 2);
add3(1, 2, 3, 4);
```

Both fail. Only `add3(1, 2, 3)` matches the three declared parameters.

`return` Stops the Body

```
function classify(n) {
    if (n < 0) { return "negative" };
    if (n == 0) { return "zero" };
    return "positive";
};
print(classify(-5));
print(classify(0));
print(classify(5));
```

Prints `negative`, `zero`, then `positive`.

Falling Off the End

```
function announce() { print("starting") };
print(announce());
```

Prints `starting`, then `null`.

`null` Is a Real Value

```
print(null);
print(null == null);
print(type(null));
```

Prints `null`, then `true`, then `null`.

Local Variables and Call Frames

```
function f(x) {
    y = x * 2;
    return y + 1;
};
print(f(5));
```

Prints 11.

Each Call Gets Its Own Frame

```
function f(x) {
    doubled = x * 2;
    return doubled;
};
a = f(3);
b = f(100);
print(a);
print(b);
```

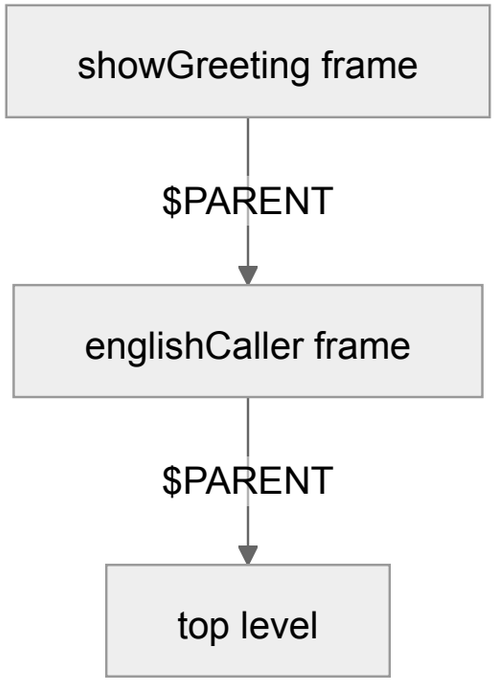
Prints `6`, then `200`.

Dynamic Scoping

```
function showGreeting() { print(greeting) };
function englishCaller() { greeting = "Hello"; showGreeting() };
function frenchCaller() { greeting = "Bonjour"; showGreeting() };
englishCaller();
frenchCaller();
```

Prints `Hello`, then `Bonjour`.

Following the `$PARENT` Chain



`showGreeting`'s frame chains back through whoever called it.

The Real Limitation

```
function makeAdder(a) {
    return function(b) { return a + b };
};
y = makeAdder(1)(2);
```

Raises `Unknown identifier: a`.

Recursion

```
function factorial(n) {
    if (n == 0) { return 1 };
    return n * factorial(n - 1);
};
print(factorial(6));
```

Prints `720`.

Calls as Expressions

```
function square(x) { return x * x };
print(1 + square(3) + 1);

function isPositive(x) { return x > 0 };
if (isPositive(5)) { print("5 is positive") }
```

Prints 11, then `5 is positive`.

Chained Calls

```
function pair() { return [10, 20] };
print(pair()[1]);

function record() { return {"value": 99} };
print(record().value);

function makeAdder() { return function(a, b) { return a + b } };
print(makeAdder()(3, 4));
```

Prints `20`, then `99`, then `7`.

Function Values: Types and Operations

Operation	Function behavior
`type(function(x) { return x })`	Returns `"function"`
`string(f)`, `print(f)`	Displays as `<function>`
`f == g`	`true` only when `f` and `g` are the same function value
Arithmetic, ordering comparisons	Type error
`length`, `head`, `tail`, `slice`, indexing	Type error

Parsing the New Syntax

```
function_expression ::= "function"
                        "(" [ identifier { "," identifier } ] ")" block
function_definition ::= "function" identifier
                        "(" [ identifier { "," identifier } ] ")" block
return_statement     ::= "return" [ expression ]
call_suffix          ::= "(" [ expression { "," expression } ] ")"
```

`call_suffix` extends the same suffix chain `[` and `.` already use.

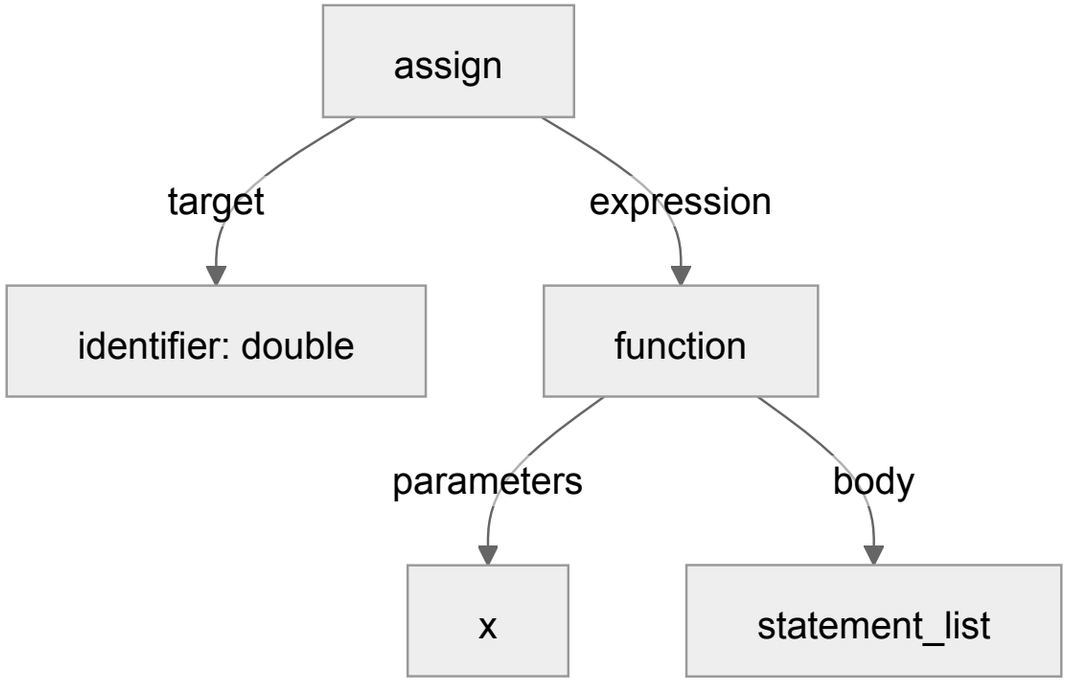
Assignment Is Now an Expression

```
expression ::= logic_or [ "=" expression ]
```

```
x = y = 1;
```

Right-associative: y is assigned first, and that assignment's own value becomes what x receives.

One AST, Named Form Resolves



`function_definition` resolves at parse time into this same tree.

Evaluation Order and Status

Body's status	Call reports
`"return"`	The returned value, with no status: the call finished normally
`None` (fell off the end)	`null`, with no status
`"exit"`	Passed outward unchanged, the same as it passes outward through a loop
`"break"` or `"continue"`	A specific error: neither one has a loop left to belong to

A New Boundary Error

```
function f() { break };
f();
```

Raises `'break' used outside a loop`.

Considerations

A function's own value retains only its parameters and body – nothing about where it was defined.

Dynamic scoping resolves a free name by looking at the caller, not at wherever the function was written.

`break` and `continue` are rejected at both boundaries; a bare `return` is rejected only at the program boundary, where there is no call left to consume it.

Working with Functions

Operation	Result
`type(function(x) { return x })`	`"function"`
`type(null)`	`"null"`
Calling with the wrong number of arguments	Runtime error
Falling off the end, or a bare `return`	`null`
`break` or `continue` with no enclosing loop	Specific boundary error